

Dziļā mašīnmācīšanās

3. mājas darbs

Arnis Priedītis

Izmantotā Jupyter notebook pieejama šeit:

<https://colab.research.google.com/drive/1q5TvhiCSt8yb3qHvxTNJDplxP7U4zhVU?usp=sharing>

Apmācību kodu uz Hemingveja latviešu valodas tulkojumu. Izdarbināju kodu ar sekojošajiem hiperparametriem:

```
batch_size = 512
block_size = 32
max_iters = 30000
eval_interval = 100
learning_rate = 1e-3
eval_iters = 200
n_embd = 256
n_head = 8
n_layer = 16
dropout = 0.2
```

Te daži no parametriem nomainīti uz tiem, kas uzdevuma aprakstā norādīts zem “Atzīmei 10”, papildus tam tikai `n_head` nomainīts no 4 uz 8. Tad arī modeļa parametru skaits palika divreiz lielāks nekā iepriekš – 12.686954 M – un katra iterācija trenēšanā aizņēma vairāk laika.

Tā kā lai gan max iterāciju skaits ir 30000, ar google colab brīvo versiju kods paspēja trenēt modeli tikai 8000 iterācijās. Zemāk redzams, kādu turpinājumu modelis ģenerē tekstam “Uz sliekšņa stāvēja”.

```
model.load_state_dict(torch.load(f"checkpoint_8000.pt"))
model.eval()

def generate_with_prompt(prompt, max_new_tokens):
    prompt_encoded = torch.tensor(encode(prompt), dtype=torch.long).unsqueeze(0).to(device)
    generated = model.generate(prompt_encoded, max_new_tokens=max_new_tokens)
    generated_text = decode(generated.squeeze().tolist())
    return generated_text

while True:
    prompt = input("Ievadi promptu vai q, lai pabeigtu sarunu: ")
    if prompt.lower() == 'q':
        break

    generated_text = generate_with_prompt(prompt, max_new_tokens=50)
    print(generated_text)
    print("-----")
```

▼ ... Ievadi promptu vai q, lai pabeigtu sarunu: Uz sliekšņa stāvēja
Uz sliekšņa stāvēja Ketrina Bārklīja.
Viņa ienāca istabā un tuvojās g

Ievadi promptu vai q, lai pabeigtu sarunu: Uz sliekšņa stāvēja
Uz sliekšņa stāvēja Ketrina Bārklīja.
Viņa ienāca istabā un tuvojās g

Ievadi promptu vai q, lai pabeigtu sarunu: Uz sliekšņa stāvēja
Uz sliekšņa stāvēja Ketrina Bārklīja.
Viņa ienāca.
"Esiet tik laipna,

Ievadi promptu vai q, lai pabeigtu sarunu: q

Trenēšanas beigu dati:

```
step 6400: train loss 0.3715, val loss 2.9097 (perplexity 18.35 / 106 = 17.31% uncertainty)
step 6500: train loss 0.3705, val loss 2.9323 (perplexity 18.77 / 106 = 17.71% uncertainty)
step 6600: train loss 0.3700, val loss 2.9382 (perplexity 18.88 / 106 = 17.81% uncertainty)
step 6700: train loss 0.3705, val loss 2.9716 (perplexity 19.52 / 106 = 18.42% uncertainty)
step 6800: train loss 0.3689, val loss 2.9846 (perplexity 19.78 / 106 = 18.66% uncertainty)
step 6900: train loss 0.3673, val loss 2.9780 (perplexity 19.65 / 106 = 18.54% uncertainty)
step 7000: train loss 0.3677, val loss 2.9764 (perplexity 19.62 / 106 = 18.51% uncertainty)
step 7100: train loss 0.3663, val loss 3.0191 (perplexity 20.47 / 106 = 19.31% uncertainty)
step 7200: train loss 0.3653, val loss 3.0302 (perplexity 20.70 / 106 = 19.53% uncertainty)
step 7300: train loss 0.3646, val loss 3.0314 (perplexity 20.73 / 106 = 19.55% uncertainty)
step 7400: train loss 0.3636, val loss 3.0276 (perplexity 20.65 / 106 = 19.48% uncertainty)
step 7500: train loss 0.3626, val loss 3.0924 (perplexity 22.03 / 106 = 20.78% uncertainty)
step 7600: train loss 0.3629, val loss 3.0681 (perplexity 21.50 / 106 = 20.28% uncertainty)
step 7700: train loss 0.3629, val loss 3.0542 (perplexity 21.20 / 106 = 20.00% uncertainty)
step 7800: train loss 0.3609, val loss 3.0883 (perplexity 21.94 / 106 = 20.70% uncertainty)
step 7900: train loss 0.3608, val loss 3.0689 (perplexity 21.52 / 106 = 20.30% uncertainty)
step 8000: train loss 0.3606, val loss 3.0715 (perplexity 21.57 / 106 = 20.35% uncertainty)
```

Start coding or [generate](#) with AI.

[Change runtime type](#)



Executing (3h 20m 26s) T4 (Python 3)